Assignment Report

Applied Deep Learning - CNN Concepts

Nisha Naresh

101008896

Ontario Tech University

AI Programming

Zahra Atf

1. What does image classification mean? Explain with one example.

The task of image classification is to label one entire image with one of the fixed set of category names learned by the model. The model processes all the picture as a whole and promises to say whether it is a bug or a dish and leaves it to the reader to draw a conclusion as to whether any of the bugs or dishes is located within or outside the box. This is different to detection that also uses a box for the objects and segmentation where each pixel is labelled.

Example: A security camera in a parking gate takes a shot and emits a car or no car for the complete image. It doesn't indicate where the car is or how large it looks, its entire frame is compressed into that one verdict which then initiates a gate. Classification has got all of this one-label-for-the-whole-image action.

2. Why is using a Dense Network not always a suitable choice for images?

In a dense fully connected structure, all the inputs to each pixel in the image are wired to all of the neurons in the next layer, leading to two significant problems. The first is a weight explosion - A medium size image of 64 color images would be a 64x64 picture, which has 64 x 64 x 3 = 12,288 inputs to pass to the network, so if the network is a single dense layer and the layer only has 500 neurons, the total number of weights would be more than 6 million already, and if there are several more layers, they are slow to train and over-fit the image. The second one is the lack of structure in space. The dense layer takes out the sense of top and bottom, and squanders the image into a row of numbers so that it doesn't recognize what rows and columns used to look like. Therefore, when the object is shifted by a pixel or two in either direction the absolute shape of the hypertextures change significantly and the network has to learn this novel pattern from scratch again. For both of these problems, convolutional layers were created; these layers utilize small filters which are applied repeatedly throughout the image.

3. In a CNN, what is convolution, and what does a filter or kernel do?

Convolution is the process of sliding a little window of weights over the image, at every point it touches, multiplying each pair of overlapping points and getting the sum, to place as one number at that point. The tiny grid itself is the filter, also known as the kernel, and it is a detector which is sensitive to one local pattern. If the filter patch matches the pattern, then the output is large; if the filter patch doesn't match, then the output remains close to zero.

Example: A horizontal-edge kernel responds strongly to brightness changes up and down along each flat step of the stairs in a photo, but is inactive on the smooth vertical wall beside the stairs. A large number of different kernels make a layer that provides a highly populated band of such detectors, which are then formed into more complex shapes by stacking additional layers.

4. What is a feature map in a convolutional neural network?

After the first filter has been convolved over the whole picture, you get a 'feature map' – this is a complete grid which gives you information on how strongly the pattern detected by that filter was found at each position. Effectively, it is a map indicating the presence (bright region) or absence (dark region) of this feature.

Example: The bicycle's feature map is darkened at the straight frame bars and handlebars, but its two round wheels glow when a circle-detecting filter is applied to a photo of the bicycle. This relationship is one-to-one; so each filter in a layer creates exactly one feature map, and the layer loaded with 16 creates 16 feature maps one on top of another; each one representing one of the learned patterns to which the subsequent layer will apply.

5. Why is weight sharing used in CNNs? Explain.

In weight sharing, the network shares its weights between all of the image positions and learns a set of common weights rather than one set for each position of the image, which would be a dense layer. The idea being that a pattern that exists in one space is a pattern that exists in any space and indeed, a shape, or a point, or a line, or a wave, is not some sort of "place" when moved. For instance, a chessboard image can have one shared (frequency-wise) filter which matches the edges of the boxes of the image at the corners and is passed through a complete sweep across the image. This means two major advantages: It greatly reduces the number of the parameters, with a single filter applied to the full image rather than having to keep different weights at all points, and provides the network with some degree of tolerance to translation (a feature would still be detected if it moved out of place).

6. How does the ReLU activation function work? Write the ReLU output for [-3, 0, 2, 5].

The Rectified Linear Unit (ReLU) is a transformation that acts on every value independently: if the value is positive, then it is passed through without modification; if the value is negative, then it is replaced by zero. If it's applied element-wise to the vector [-3, 0, 2, 5], the negative term is made 0, the 0 remains 0, the 2 becomes 2, and the 5 remains 5. For most of the CNNs, ReLU is the default choice for three reasons. It provides non-linearity; the network may not model only linear relationships, but more complex ones as well. It is an extremely inexpensive calculation, a comparison to zero. So it allows gradients to easily propagate through positive inputs in training, unlike previous functions such as sigmoid which slowed them down, which accounts for the use of very deep networks.

7. What is max pooling, and why is it used in CNNs?

Instead of Max pooling, a feature map is divided into small non-overlapping regions, usually 2x2, and only the maximum value, which is the largest number in that particular region, is preserved where the rest are discarded. This results in a smaller map showing the strongest signals. Example: a 2x2 window applied to the patch [[7, 1], [3, 5]] returns 7, throwing away the 1, 3, and 5. It will be used for 3 related reasons. It increases the compression of the information, thus lowering the amount of processing power, size of computations and memory used by subsequent layers. It saves the most prominent activations, which it does not average down the way that a standard average would. And it includes very mild translation invariance, since a pixel is moved just one pixel into a feature will typically be inside the same "pool" and so get the same pooled output, so later decisions are more "translation" invariant against small pixel motions.

8. In the Fashion-MNIST code, why are images reshaped from 28x28 to 28x28x1?

In machine learning frameworks such as Keras, convolutional layers do require the input to explicitly include an explicit channel dimension (as (height, width, channels)). Grayscale images, such as those in Fashion-MNIST, have a single channel because each pixel in the image is a single level of brightness (not a set of three colors). This raw data is saved in the flat 28x28 shape, without an axis for the channels, hence the reshape adds 1 at the end to create 28x28x1, giving the same shape the layer expects. This does not affect the actual situation—no values in any pixel are modified, no additional information is provided—it merely re-formats the picture to make the shape match the picture's requirement. In contrast, a "color" image would sprawl to 3, one channel each for red, green and blue, so the single 1 is just the network's representation of "this is one channel grayscale".

9. What is the difference between training data, validation data, and test data?

Each set serves a different purpose and separation of the sets is achieved by their degree of exposure to each set. The model is trained on a set of data called training data and only the examples in the training set update the model's weights. The model is never used to change weights, but is validated against the data while training to make decisions as when to stop training, which settings to keep, etc., so the model (indirectly) satisfies it because of your decisions. Test data is never used at all, it's only opened at the very end to get a fair comparison of the model's performance on data it has never changed. Example: Train a digit reader on 60,000 samples; it will learn the shape; then on the reserved 5,000 samples, check its accuracy; if it fails, it stops training before it experiences the problem of overfitting; once it has completely ended training, it shows the results of the accuracy check on a completely new and unobserved 5,000. If you tuned an instrument against this last set the last number would not be honest which is why the three are kept separated.

10. What is overfitting? If training accuracy is high but validation accuracy is low, what does this indicate?

Overfitting occurs when a model "memorizes" the training data, including irrelevant noise, rather than the general underlying trends that extend into new data. An overfit model gives great accuracy on what it has learned, but poor accuracy on what it hasn't learned. It's the signature that you'll see in the textbook: good training accuracy and poor validation accuracy, the network has learned exactly how to do its task, but not learned the structure for generalization. In an example, the accuracy of a sign-language classifier improves with training data when the data is from a subset of its own population, but it drops to 70 percent on an outside sample of data. That 28-point difference speaks of the fact that it learned the gestures, but in the lights and the layout of the pixels of those pictures—it did not learn the gesture shape. One of the typical remedies are closing that gap with more data, dropout, or regularization.